

Oak Network Smart Contracts Audit



June 17, 2026

Table of Contents

Table of Contents	2
Summary	5
Scope	6
System Overview	7
Core Protocol Modules	7
Treasury Models	8
Shared Utilities	9
Security Model and Trust Assumptions	9
Privileged Roles	10
Trusted Forwarder	12
Integration Considerations	13
Refactoring Opportunities	14
Critical Severity	16
C-01 Approvals Can Be Abused to Spend Others' Funds	16
Medium Severity	17
M-01 Cancellation in PaymentTreasury Allows Post-Cancel Admin Sweeping While Disabling Refunds	17
M-02 Fee Types Mixing in Storage With Lack of Validation	18
M-03 disburseFees Is Vulnerable to Reentrancy	19
M-04 Campaign Cancellation Does Not Start Refund Schedule	19
M-05 disburseFees and Withdraw Function Override Removes Constraints	20
M-06 Incorrect Modifier	20
M-07 removePlatformData Does Not Check That platformDataKey Corresponds to platformHash	21
M-08 Assets Might Get Permanently Blocked After cancelTreasury in KeepWhatsRaised	21
Low Severity	22
L-01 Treasury Can Get Attached to Expired Campaign	22
L-02 Burn Function Publicly Callable	23
L-03 Inconsistent Colombian Tax Accounting	23
L-04 BaseTreasury.disburseFees Can Be Called Multiple Times	24
L-05 Withdraw Can Happen Before Refund Window Ends	25
L-06 setFeeAndPledge Has No Time Restrictions	25
L-07 Deselecting Platforms Skips platformData Setting	26
L-08 Unescaped s_imageURI Can Produce Invalid tokenURI JSON Metadata	26
L-09 Rewards Can Be Duplicated	27

L-10 Rewards Can Be Removed	27
L-11 Rewards for Pledge May Conflict	28
L-12 setFeeAndPledge Combines Different Flows	28
L-13 Old Project Name Used for Storage Slots	29
L-14 Lack of Checks When Setting campaignData	29
L-15 Unreachable Branch	30
L-16 Missing Check in _updateFiatTransaction	30
L-17 configureTreasury Can Be Called Multiple Times	31
L-18 Deprecated Platform Data Due to removePlatformData Cannot Be Performed After Platform Is Delisted	31
L-19 delistPlatform Does Not Clear platformAdapter Mapping Entry	32
L-20 Storage Update Without Emitting Event	32
L-21 Possible to Add Same Token to acceptedTokens List Twice	32
L-22 Fee Values Are Uncapped	32
L-23 Not Checking s_fundClaimed	33
L-24 _msgSender Overloading in Internal ID System Could Cause Accounting Errors	33
L-25 Unbounded Arrays As Inputs	34
L-26 Shipping Fee Not Verified	35
L-27 Platform Data Passed Might Not Equal Amount of Platforms Passed	36
L-28 Missing Initializer Lock in Treasury Implementations	37
L-29 Lack of Documentation About the backer	37
L-30 Platform Data Keys Are Not Bound to the Updated Platform During Selection Updates	38
L-31 Platform Adapter Rotation Does Not Revoke the Trusted Forwarder in Deployed Treasuries	38
L-32 Missing Validation	39
L-33 Inconsistent Use of paymentId	39
L-34 Mismatch in Fee Storage	40
L-35 Treasury Self-Pledge Misuses NFT Check	40
Notes & Additional Information	41
N-01 Redundant Conditionals	41
N-02 Reverts With Multiple Conditions	42
N-03 Unnecessary Temporary Variables	42
N-04 Unnecessary Assignment in AllOrNothing._pledge	43
N-05 external Function With internal Naming Convention	43
N-06 Multiple Event Emissions for Same Assignment	44
N-07 addItem And addItemBatch Functions Can Modify Existing Items	44
N-08 Non-Reward Pledges Are Not De-Normalized	44
N-09 Unnecessary Override	45
N-10 Renaming Opportunities	45
N-11 Unnecessary Assignment Wastes Gas	46
N-12 Unclear Docstrings	46
N-13 Not Using Require With Custom Errors	47
N-14 Unchecked Scoping Not Needed for for Loop Iterators	48
N-15 Structs Declared in Middle of Contracts	48
N-16 Implicit Returns and Unnamed Return Values	48
N-17 Missing Documentation	49
N-18 Unused Code	49
N-19 Repeated Use of Errors	50
N-20 Typographical Error	50

Conclusion	51
Appendix	52
Issue Classification	52

Summary

Type	Contracts	Total Issues	64 (49 resolved, 6 partially resolved)
Timeline	From 2026-01-05 To 2026-02-04	Critical Severity Issues	1 (1 resolved)
Languages	Solidity	High Severity Issues	0 (0 resolved)
		Medium Severity Issues	8 (8 resolved)
		Low Severity Issues	35 (24 resolved, 4 partially resolved)
		Notes & Additional Information	20 (16 resolved, 2 partially resolved)

Scope

OpenZeppelin performed an audit of the [oak-network/contracts](#) repository at commit [479241c](#).

In scope were the following files:

```
src
├─ CampaignInfo.sol
├─ CampaignInfoFactory.sol
├─ GlobalParams.sol
├─ TreasuryFactory.sol
├─ constants
│   └─ DataRegistryKeys.sol
├─ interfaces
│   ├── ICampaignData.sol
│   ├── ICampaignInfo.sol
│   ├── ICampaignInfoFactory.sol
│   ├── ICampaignPaymentTreasury.sol
│   ├── ICampaignTreasury.sol
│   ├── IGlobalParams.sol
│   ├── IItem.sol
│   ├── IReward.sol
│   └─ ITreasuryFactory.sol
├─ storage
│   ├── AdminAccessCheckerStorage.sol
│   ├── CampaignInfoFactoryStorage.sol
│   ├── GlobalParamsStorage.sol
│   └─ TreasuryFactoryStorage.sol
├─ treasuries
│   ├── AllOrNothing.sol
│   ├── KeepWhatsRaised.sol
│   ├── PaymentTreasury.sol
│   └─ TimeConstrainedPaymentTreasury.sol
└─ utils
    ├── AdminAccessChecker.sol
    ├── BasePaymentTreasury.sol
    ├── BaseTreasury.sol
    ├── CampaignAccessChecker.sol
    ├── Counters.sol
    ├── FiatEnabled.sol
    ├── ItemRegistry.sol
    ├── PausableCancellable.sol
    ├── PledgeNFT.sol
    └─ TimestampChecker.sol
```

System Overview

Oak Network is programmable commerce and escrow infrastructure - an on-chain backbone for creating and managing conditional payment flows. The repository highlights support for cross-listable payment campaigns, multiple treasury models for holding and releasing funds, programmable NFT-based receipts (one ERC-721 collection per campaign), ERC-2771 meta-transactions for platform-admin operations, and UUPS upgradeability for core contracts.

At a high level, the system bifurcates its logic in the following manner:

- **Campaign identity and configuration:** What the campaign is, who administers it, what it is selling/offering, what rules apply.
- **Funds custody and settlement:** How money is collected, when it can be withdrawn, and under what conditions refunds occur.

This separation allows the protocol to support multiple funding/settlement “treasury models” while keeping campaign metadata and governance consistent across platforms and integrations.

Core Protocol Modules

GlobalParams

The `GlobalParams` contract is the protocol’s central configuration point. It manages protocol-wide parameters and shared configuration needed by the rest of the system, such as currency/token configuration, platform-level parameters, and fee configuration.

Conceptually, the `GlobalParams` contract enables a deployment to be operated as a shared piece of infrastructure: platforms can be registered/configured once, and the rest of the system can enforce consistent rules and integrations across all treasuries and campaigns.

CampaignInfo and CampaignInfoFactory

The `CampaignInfoFactory` contract is the entry point for campaign creation and management, and the `CampaignInfo` contract represents the per-campaign on-chain object. `CampaignInfo` supports structured items and rewards, enabling platforms to present standardized campaign offerings.

Campaigns are designed to be represented once on-chain and referenced by multiple platforms, reducing fragmentation where each platform would otherwise maintain its own separate campaign contracts and state. To do so, each campaign is deployed as a clone through the factory, and treasuries are attached/allowed to the campaign as part of the flow.

TreasuryFactory

TreasuryFactory is responsible for deploying the campaign treasury contracts (the custody/settlement layer). Treasury implementations are registered and approved per platform, after which treasuries can be deployed for campaigns.

This factory-based approach gives platforms flexibility to choose treasury models per campaign (and potentially evolve their implementations over time), while also centralizing the allowed implementation set into a single data location.

Treasury Models

The available treasuries consist of concrete fund-management strategies:

- **AllOrNothing** : A crowdfunding-style escrow where funds are refunded if a goal is not met.
- **KeepWhatsRaised** : A flexible model where funds can be kept regardless of whether a goal is met, while tips and configurable fees/withdrawal gating can also be included.
- **PaymentTreasury** : A “payment-style” treasury oriented around off-chain payment creation and on-chain confirmation, including support for line items and optional NFT minting.
- **TimeConstrainedPaymentTreasury** : A time-gated **PaymentTreasury** variant constrained by a time window.

Each one of these treasuries has common functionalities that work alongside the **CampaignInfo** contracts to query and retrieve information such as the total raised or the parameter configuration.

Note that these treasuries allow spending another wallet's tokens to mint pledge NFTs. This is an important security consideration. Thus, allowances to treasuries should be given and revoked with special attention to avoid loss of funds.

Shipping Fees and Tips

The protocol implements the concept of "shipping fees" and "tips" which are included when making pledges for crowdfunding campaigns. Importantly, shipping fees and tips are not included in calculations for percentage-based or flat fees for the protocol, or in calculations of the total amounts raised. Shipping fees and tips may provide a way to bypass certain accounting mechanisms and may allow users to discreetly transfer extra funds to the campaign admins.

Shipping fees are intended to cover the costs of shipping a physical reward to the user providing funds, but they are generally unchecked and unbounded.

Shared Utilities

Aside from the aforementioned contracts, the protocol also implements other contracts for consistent enforcement of permissions, lifecycle controls, and upgrade safety processes. These include:

- `AdminAccessChecker` and `CampaignAccessChecker` for access controls
- `PausableCancellable` for runtime controls
- `TimestampChecker` for time validation, enforcing deadlines for certain actions
- `PledgeNFT` for campaign-level NFTs
- storage libraries for modular, [ERC-7201](#)-compliant storage isolation

Security Model and Trust Assumptions

This system's security depends on a set of explicit governance and integration assumptions:

- **Upgrade authority is trusted:** Any account (or governance process) empowered to authorize upgrades can materially change protocol behavior, including fund custody logic via factories/registries, so compromise or misuse of upgrade power is a systemic risk.
- **Platform administrators are trusted operators:** The protocol is designed for "multi-platform" use. This implies platform admin keys (and their meta-tx forwarding setup) are

expected to be secure and behave correctly; malicious or compromised platform admin control could impact campaign operations on that platform.

- **Treasury implementation governance is security-critical:** The repository's deployment workflow describes registering and approving treasury implementations *per platform*. This means the system inherently trusts the process that selects/approves implementations, since a malicious treasury implementation can directly threaten user funds.
- **Off-chain components (especially for `PaymentTreasury` flow):** The `PaymentTreasury` uses off-chain payment creation with on-chain confirmation. Correctness of itemization, pricing, and the mapping between off-chain intent and on-chain settlement is an implicit trust assumption for integrators using that model.
- **Platform administrators are trusted to behave in a way that respects their users:** For certain treasuries, admins are allowed to withdraw funds after users deposit, which may interfere with users' abilities to obtain refunds. Platform admins are trusted not to interfere with the refund process. Platform admins are also trusted to broadcast and confirm all off-chain payments on behalf of users. It should be noted that because all funds are accounted for within the same "buckets", it is possible that users may experience race conditions when attempting to get refunds. If there are not enough funds available for refunding every user, the last users to call for a refund will be denied.
- **Platform administrators are trusted to resolve any disputes regarding off-chain and on-chain funding:** For example, they are trusted to relay all off-chain payments to the blockchain. They are trusted to do so within the required time windows (before campaign expiration) and trusted to only relay off-chain payments that are successful, versus those that may fail due to credit-card chargebacks or other reasons. Moreover, they are trusted to not mistakenly trigger an on-chain cancellation of an off-chain payment if it had been genuinely made.

Privileged Roles

While exact function-level permissions are implemented in-code (notably via `AdminAccessChecker` / `CampaignAccessChecker`), the architecture implies the following privilege tiers:

In `CampaignInfo`:

- Owner (campaign owner) can transfer the ownership, update launch time, deadline, goal amount, and selected platforms (pre-launch/when not locked), and update NFT image/contract metadata (pre-launch).
- Protocol admin can pause and unpause the campaign.

- Protocol admin or owner can cancel the campaign.
- Treasury factory address can approve a platform treasury, store its address, grant MINTER_ROLE to that treasury, and lock the campaign.
- MINTER_ROLE (meant for the approved treasuries) can mint pledge NFTs when the backers pledge.

In `CampaignInfoFactory`, Owner (factory owner) can update the campaign implementation and authorize UUPS upgrades.

In `GlobalParams`:

- Owner (protocol admin at init) can authorize upgrades, add registry entries, enlist/delist platforms, update protocol admin address, update protocol fee percent, update platform admin address, set platform adapter, and add/remove tokens for currencies.
- Platform admin (per platform) can add/remove platform data keys, update platform claim delay, set platform line-item types, and remove line-item types.

In `TreasuryFactory`:

- Protocol admin can approve/disapprove treasury implementations and authorize UUPS upgrades.
- Platform admin (per platform) can register/remove treasury implementations for their platform and deploy a treasury (which also sets platform info in the campaign).

In `PledgeNFT`, the MINTER_ROLE can mint pledge NFTs.

In `BaseTreasury`, the platform admin can pause, unpause, and cancel a treasury.

In `BasePaymentTreasury`:

- Platform admin can create off-chain payment records, confirm off-chain payments, cancel payments, claim refunds for off-chain payments, claim non-goal line items, claim expired funds, and pause/unpause/cancel the treasury.
- Platform admin or campaign owner can withdraw campaign funds.

In `AllOrNothing`:

- Campaign owner can add and remove rewards.
- Platform admin or campaign owner can cancel the treasury.
- Platform admin can pause/unpause the treasury.

In `KeepWhatsRaised`:

- Platform admin can set payment gateway fees, approve withdrawals, configure treasury parameters, execute `setFeeAndPledge`, claim tips, claim remaining funds, and pause/unpause.
- Campaign owner can add and remove rewards.
- Platform admin or campaign owner can update deadline and goal amount, withdraw funds, and cancel the treasury.

In `PaymentTreasury` and `TimeConstrainedPaymentTreasury`, the platform admin or campaign owner can cancel the treasury.

Trusted Forwarder

In order to enable certain transactions on behalf of admins or multisigs, an out-of-scope contract was created called the trusted forwarder. This contract is central to the usage of `_msgSender()` in the codebase, instead of `msg.sender`.

When the `msg.sender` within the codebase is the "Trusted Forwarder", the `_msgSender()` will be considered the last 20 bytes of the `msg.data`. So, the trusted forwarder appends the "caller" address to its transactions, and this is used by contracts like `BasePaymentTreasury` to credit accounts or for access control.

It is assumed that the trusted forwarder is programmed correctly, and that it is impossible to have an address which is not the user's appended to the `msg.data` when the trusted forwarder forwards calls into the audited codebase. If the trusted forwarder were to break, either not appending addresses to the calldata or appending incorrect addresses to the calldata, many problems could arise. For example, funds could be credited to inaccessible accounts, or users without permissions could gain access to sensitive admin functions, potentially transferring protocol funds to themselves or changing system parameters in a dangerous way.

Integration Considerations

Projects integrating with Oak Network contracts (platforms, frontends, indexers, and external protocols) should account for the following:

- **Treasury models do not all behave alike:** Different campaigns may use different treasuries with different settlement rules. Integrators should avoid assuming a single “finalize/withdraw/refund” flow for all campaigns and instead implement different logic based on the treasury type.
- **Per-campaign NFT collections:** The system supports campaign-level Pledge NFTs. Indexers and marketplaces should expect many ERC-721 collection addresses rather than a single global collection, and should treat each campaign’s pledge NFT contract as a distinct collection.
- **Currency abstraction and multi-token mappings:** Currency-based multi-token campaigns imply that the protocol distinguishes between a currency identifier and one or more ERC-20 token addresses used to pay in that currency. Integrators must ensure they select a token that is actually configured/allowed for the relevant currency/platform and that they use the correct unit conventions.
- **Meta-transaction awareness (ERC-2771):** Platforms using the meta-transaction pathway need to ensure their relayers/forwarders are configured correctly and that any off-chain tooling understands that the effective actor may be `_msgSender()` in ERC-2771 contexts. Incorrect assumptions here can lead to authorization failures or operational mistakes.
- **There are multiple "pause" and "cancellation" features:** The `whenNotPaused`, `whenNotCancelled`, `whenCampaignNotPaused`, and `whenCampaignNotCancelled` modifiers all rely on different storage variables, and gate the functionality of different functions within different treasuries. Integrators should check and carefully monitor all relevant storage values related to pausing and cancellation as needed for their purposes.
- **Treasuries possess 2 sets of modifiers:** These modifiers validate the cancelled and paused state of the treasuries and the campaign. However, they are not linked to each other. This means that one module can be in a particular state, but it does not imply the rest is in the same state.

Refactoring Opportunities

The audit identified a few instances that can benefit from refactoring. While these are mentioned for consideration, they are not expected to be solved for the "fix review" portion of the audit. This is also not an exhaustive list, and there are many similar patterns that exist in multiple places within the codebase.

- The `confirmPayment` and `processCryptoPayment` functions of the `BasePaymentTreasury` contract share similar logic. Similarly, the `_updateLineItemsForConfirmation` and `_calculateLineItemTotals` functions can likely be modified to be re-used, ensuring that logic is consistent between the two.
- The `BaseTreasury` contract is inherited by the `AllOrNothing` and `KeepWhatsRaised` contracts. It defines functions that are also defined in `BasePaymentTreasury`. Consider implementing a contract inherited by both `BaseTreasury` and `BasePaymentTreasury` which defines functions, like `pause` and `cancelled`, that are already shared by the two existing contracts with nearly identical definitions, which would simplify the maintenance of the code and increase the readability. Consider also moving the definitions of the `disburseFees` and `withdraw` functions of the `BaseTreasury` contract into the `AllOrNothing` contract since that is the only place they are used (these functions are overridden in the `KeepWhatsRaised` contract). This will make the logic in the `AllOrNothing` and `KeepWhatsRaised` contracts clearer, and keep logic inside the `abstract BaseTreasury` contract minimal.
- Consider implementing the "normalization" and "denormalization" functionalities within a library, where rounding can be specified. This will ensure that behavior is predictable and consistent in all contexts.
- Consider combining the `whenNotPaused`, `whenNotCancelled`, `whenCampaignNotPaused`, and `whenCampaignNotCancelled` modifiers into single modifiers that can be used in place of all 4, or common groupings of 2. This will prevent one from being forgotten on a function definition, or other simple mistakes. Ensure that such modifiers are placed on external entry points rather than on internal functions.
- "batch" functions should generally make use of the non-batch equivalent function or its internal logic. Consider refactoring "batch" functions to be very minimal, relying on the non-batch versions. This will ensure that behavior is identical between the two.
- The frequent `checking of the "internalId" for payments` can be extracted into a function. Computing then checking the internal ID for both off-chain and on-chain can be done in

one function. In addition, the `_scopePaymentIdForOnChain` function can use the "creator" address as an input parameter instead of `_msgSender`, allowing it to be used in place of `inline hashing`, which happens frequently within the `BasePaymentTreasury` contract. Reverting when detecting an already-used payment ID can be extracted into a function. These instances are frequent and can be found by searching for the `PaymentTreasuryPaymentAlreadyExist` error.

- Consider redefining `TimestampChecker` as a library, and moving functions from the `TimeConstrainedPaymentTreasury` contract like `_checkTimeWithinRange` and `_checkTimeIsGreater` functions into the `TimestampChecker` library, potentially as modifiers.
- The `_checkSuccessCondition` function is largely irrelevant for most treasuries except the `AllOrNothing` contract. Consider scoping it locally to that contract, implementing it within `AllOrNothing`, and using it within the local versions of `withdraw` and `disburseFees` functions. Consider deleting all other references to it.

Cleaner code is easier to audit and maintain, and less error-prone. Generally, re-using code in the form of libraries, internal functions, and modifiers will help keep development functioning smoothly. It will make the code easier to reason about and extend, as well as keeping lines-of-code low. Most importantly, it will ensure consistent behavior and make updates to any major functionality significantly easier.

When modifying the code for substantial refactors, ensure that all test coverage passes before integration. Identify gaps in test coverage, including edge cases, and increase test coverage during refactors to ensure no missed changes in behavior between the old and new version. It is recommended to conduct a re-audit with a reputable firm after any substantial refactoring.

Critical Severity

C-01 Approvals Can Be Abused to Spend Others' Funds

The `processCryptoPayment` function of the `BasePaymentTreasury` contract is callable by anyone, and the `buyerAddress` and `lineItems` array arguments are passed in as defined by the user. At the end of the function, a `safeTransferFrom` call is performed, transferring assets from the `buyerAddress` to the `BasePaymentTreasury` contract. The `totalAmount` transferred is the `passed-in amount` plus the sum of all "line item" amounts. Although an NFT is minted to the `buyerAddress`, the amount associated with it will be the `passed-in amount`.

This means that any time a user approves the `BasePaymentTreasury` contract for some token, any address can call this function and drain any approved amount from their address. They can modify the intended `amount` such that the amount associated with the NFT is extremely low, and the amounts associated with the line items are very high. This can be abused by the campaign creator to receive excess funds, either due to the buyer approving too much, the token approvals not decreasing as they are spent (in this case, the attack could be performed multiple times quickly), or through the caller creating a `lineItems` array which consumes most of the funds, leaving the amount associated with the NFT very small. This attack could also be performed for the sake of griefing, and may also be incentivized by a campaign supporter who wants to see the campaign's fundraising targets be achieved.

To perform this attack, the attacking account simply has to watch the mempool for approvals, and then craft transactions that spend those approvals via calls to `processCryptoPayment` function. Similarly, within various other treasury contracts, pledges can also be made with tokens coming from a different account than `_msgSender`. The `AllOrNothing._pledge` function and the `KeepWhatsRaised._pledge` function both support minting pledge NFTs with tokens transferred from some user-specified address. Due to the usual non-atomicity of the operation, there will be a delay that allows a malicious actor to create a transaction which spends those tokens without the original user's consent. In particular, the malicious user could use the funds originally meant for a pledge with reward for a pledge without reward.

Consider restricting calls to the `processCryptoPayment` function such that the buyer must also be the message sender or an authorized caller. Alternatively, consider implementing some other check to ensure that the caller has permission to define line items for the buyer.

Update: Resolved at [commit cfa314e](#). All user-facing token transfer functions now use Permit2 `permitWitnessTransferFrom` instead of direct `safeTransferFrom`. The token owner must sign an EIP-712 witness committing to all critical parameters (amounts, line items, reward selections), preventing any third party from spending approved funds or altering transaction parameters. `KeepWhatsRaised` pledge IDs are now globally unique rather than per-caller scoped. The admin-only `setFeeAndPledge` path retains direct ERC20 transfers from the caller's own balance.

Medium Severity

M-01 Cancellation in `PaymentTreasury` Allows Post-Cancel Admin Sweeping While Disabling Refunds

In the logical flow of the `PaymentTreasury` and `TimeConstrainedPaymentTreasury` contracts, cancellation is triggered through the call to the `cancelTreasury` function, which sets the local treasury to cancelled via the `_cancel` function in the `PausableCancellable` contract. After that event, the refund paths through the `claimRefund` functions become unavailable because they are protected by the `whenNotCancelled` modifier.

However, during the same post-cancellation stage, admin functions such as `claimExpiredFunds` and `claimNonGoalLineItems` remain callable because they do not enforce the `whenNotCancelled` modifier, which allows the platform admin to collect the funds. This creates a deviation in cancellation expectation where, after cancellation, users can be blocked from refund execution while the platform can still extract balances that include refundable portions, provided the claim window has opened and the treasury still holds those funds. The result is a potential for abuse of user funds by administrators.

Consider enforcing a single cancellation settlement policy across all payment treasuries, where cancellation either preserves an explicit refund-first window and blocks admin sweeping until that window ends, or preserves admin sweeping but does not disable refunds immediately upon cancellation. Alternatively, consider excluding refundable accounting buckets from the

`claimExpiredFunds` function call and other admin-claim routes while cancellation refund rights are active.

Update: Resolved at [commit e5745cd](#). The modifiers preventing users from getting refunds after treasury cancellation have been removed. It is worth noting that the `claimNonGoalLineItems` function can be called during the window before the expired assets can be claimed after cancellation.

M-02 Fee Types Mixing in Storage With Lack of Validation

In the `KeepWhatsRaised` contract, the `configureTreasury` function stores [flat fees and percentage fees](#) in the same mapping (`s_feeValues`) using arbitrary keys:

- Flat fees:

- `s_feeValues[feeKeys.flatFeeKey] = feeValues.flatFeeValue`
- `s_feeValues[feeKeys.cumulativeFlatFeeKey] = feeValues.cumulativeFlatFeeValue`

- Percentage fees:

- `s_feeValues[feeKeys.grossPercentageFeeKeys[i]] = feeValues.grossPercentageFeeValues[i]`

This creates potential for error when querying a value from the `s_feeValues` mapping, as units and how to use their value are not explicitly specified.

Secondly, the function only validates array-length parity and does not validate:

- key uniqueness across flat and percentage fee keys
- per-fee percentage bounds (each fee should be less than `PERCENT_DIVIDER`)
- aggregate percentage bounds (total fee percent should be less than `PERCENT_DIVIDER`)

Since keys are stored in a shared `bytes32 => uint256 mapping`, a repeated key can overwrite another fee category. The overwritten value may then be interpreted with a different unit depending on context. For example, as a token amount during withdraw or as basis points in fee computation.

Consider enforcing key uniqueness in `configureTreasury`, enforcing percentage constraints as mentioned above, and separating fee types into different storage variables.

Update: Resolved at [commit b6dd085](#).

M-03 `disburseFees` Is Vulnerable to Reentrancy

The `disburseFees` function of the `BaseTreasury` contract makes external calls in the form of `token transfers`. Depending on the token's contract code, these calls could trigger re-entrant calls to the `disburseFees` function, which will transfer fees again. In the worst case, the contract could be nearly entirely drained, transferring tokens to the protocol admin and platform admin addresses.

Consider implementing `nonReentrant` modifiers on the `disburseFees` function. In addition, consider scanning tokens for potential re-entrant capabilities within their `transfer` functions. Note that some token contracts may be upgradeable, so their behavior may change over time.

Update: Resolved at [commit 0cc355c](#).

M-04 Campaign Cancellation Does Not Start Refund Schedule

The protocol implements the `PausableCancellable` contract to signal that a contract is either paused or cancelled, allowing or disallowing actions with the usage of the modifier. In the protocol, there are 2 distinctive modules operating as such: the `CampaignInfo` contract and the treasury contracts. These states are independent of each other, but treasury contracts [make use of both sets of modifiers](#) to restrict the operation of certain functions when these events happen.

However, while a campaign cancellation is done through the `_cancelCampaign` function, the `KeepWhatsRaised` contract stores a local cancellation marker, `s_cancellationTime`, which is only written in the `cancelTreasury` function. This means that when a `refund` is expected due to the campaign cancellation, the `_checkRefundPeriodStatus` function does not consult the `INFO.cancelled` status and considers the treasury cancelled only when `s_cancellationTime > 0`. As a result, cancelling only the `CampaignInfo` contract keeps refunds on the non-cancelled schedule and requires `block.timestamp > getDeadline`, delaying refunds until the original deadline even though the campaign is already cancelled. This is a refund liveness failure and can be used to lock backer funds for the remaining campaign duration.

Consider unifying cancellation semantics by treating `INFO.cancelled()` as cancellation for refund scheduling, or by making `CampaignInfo` cancellation automatically trigger

`cancelTreasury` on all associated treasuries. Additionally, consider unifying all the states of the cancellable and pausable contracts to prevent state gaps during operation.

Update: Resolved at [commit 2ce3f1e](#). A new effective cancellation time has been implemented which is the lowest of both.

M-05 `disburseFees` and `Withdraw` Function Override Removes Constraints

The `disburseFees` and `withdraw` functions of the `KeepWhatsRaised` contract override the inherited function from the `BaseTreasury` contract. In particular, the `BaseTreasury` contract version has the `whenCampaignNotPaused` and `whenCampaignNotCancelled` modifiers attached to those functions, which the version in the `KeepWhatsRaised` contract lacks. Due to the override, these modifiers will never be called nor will restrict the operation under such scenarios.

Consider implementing these modifiers in the `KeepWhatsRaised` contract version of the aforementioned functions.

Update: Resolved at [commit c68b28c](#).

M-06 Incorrect Modifier

In the `KeepWhatsRaised` contract, the `withdraw` function allows withdrawals when the current timestamp is less than `deadline + withdrawalDelay`. However, the `withdrawalDelay` value is documented as "Time delay (in timestamp) enforced before a withdrawal can be completed", which seems to be indicating that it cannot be withdrawn before that time.

Consider replacing the `currentTimeIsLess` modifier with the `currentTimeIsGreater` modifier to match intended behavior. Alternatively, if `currentTimeIsLess` is the desired behavior, consider fixing [the documentation](#) to reflect this.

Update: Resolved at [commit c0b778b](#). Documentation has been updated.

M-07 `removePlatformData` Does Not Check That `platformDataKey` Corresponds to `platformHash`

The `GlobalParams` contract allows the platform admin to remove data keys using the `removePlatformData` function. However, even though the function does check that the one calling is the `admin for the specified platformHash`, it does not check that the `platformDataKey` value in fact belongs to the `platformhash` value. This means that any platform admin could remove platform data at will from any other platform.

Consider asserting that the `platformDataKey` value to be removed corresponds to the specified `platformHash` value.

Update: Resolved at [commit de16755](#).

M-08 Assets Might Get Permanently Blocked After `cancelTreasury` in `KeepWhatsRaised`

The `KeepWhatsRaised` contract tracks protocol and platform fees per accepted token during pledging. The fee buckets are credited in `_calculateNetAvailable`, while the corresponding ERC-20 balances remain held by the treasury contract until fees are transferred out via `disburseFees`. However, `disburseFees` is protected by `whenNotCancelled`, while `cancelTreasury` is callable by either the platform admin or the campaign owner via `onlyPlatformAdminOrCampaignOwner`. Since cancellation is irreversible, cancelling a treasury permanently disables the only code path that zeroes and transfers `s_platformFeePerToken` and `s_protocolFeePerToken`. As a result, accrued protocol and platform fees become unrecoverable and remain stranded in the treasury contract.

Moreover, the `claimFund` function is meant for the platform admin to collect the remaining funds from the campaign after a time window. During the allowed window, the implementation transfers the balance in the `s_availablePerToken` mapping for each token. However, the contract might have a non-accounted-for leftover balance that will not be transferred during the cancellation.

Consider allowing fee disbursement after cancellation by removing `whenNotCancelled` from `disburseFees`, or adding a dedicated post-cancel fee sweep that can transfer and zero `s_platformFeePerToken` and `s_protocolFeePerToken`. Consider also restricting cancellation authority if the protocol relies on cancellation for recovery rather than for discretionary owner control. Furthermore, consider allowing a sweep mechanism after

cancellation plus some time delay, to prevent having stuck assets in the contract. Note that these assets may come from accidental sends to the contract.

Update: Resolved at [commit 270f7e7](#). The `whenNotCancelled` modifier has been removed from the function.

Low Severity

L-01 Treasury Can Get Attached to Expired Campaign

During treasury onboarding, the `TreasuryFactory` contract executes the `deploy` function, which [calls the `CampaignInfo` contract through the `\_setPlatformInfo` function](#) to approve and attach a platform treasury for a campaign. In this flow, the [_setPlatformInfo function checks](#) authorization, platform selection, approval status, and pause state, but it does not verify whether the campaign timeline has already expired. This means a new treasury can still be connected after the campaign deadline has passed and it is no longer active.

At the same time, campaign locking depends on the `s_isLocked` variable, which is only [set after the first successful call](#) to the `_setPlatformInfo` function. Until that first treasury is attached, the campaign owner can still call the [updateLaunchTime function, the updateDeadline function, and the updateGoalAmount function](#) because they are guarded by the `whenNotLocked` modifier. The deviation is that the `updateDeadline` function does not require the new deadline to be in the future relative to the current block time, while the `updateLaunchTime` function only requires the updated launch time to be in the future at call time and consistent with minimum duration. Under the condition that a campaign is created but no treasury is ever attached, an expired campaign can be revived by first moving the `deadline` to the future and then moving the `launchTime` to the future, after which treasury deployment can proceed. This can undermine timeline integrity and allow post-expiry reconfiguration of core campaign terms, which can create inconsistent campaign state and trust assumptions for participants and integrators.

Consider enforcing a campaign lifecycle check in the `_setPlatformInfo` function so treasury attachment is rejected once the campaign has passed an explicit time boundary and when the campaign is cancelled. Alternatively, consider enforcing the same lifecycle gate in the `deploy` function of the `TreasuryFactory` contract before treasury initialization and before calling the `_setPlatformInfo` function. Additionally, consider restricting the

`updateLaunchTime` function, the `updateDeadline` function, and the `updateGoalAmount` function to a pre-launch phase and aligning their time constraints with the creation-time timing policy.

Update: Resolved at [commit d0c7a18](#) and [commit 770564e](#). Checks have been added in the `updateLaunchTime`, `updateLaunchTime`, and `updateGoalAmount` functions to prevent updating after it was already launched, and now the `setPlatformInfo` function cannot be executed if the campaign is cancelled.

L-02 Burn Function Publicly Callable

When a backer pledges through a treasury, it will end up [creating an NFT in the CampaignInfo contract](#). During a refund, the treasuries [burn this NFT](#) and then return the funds. However, because the NFT is from the common `CampaignInfo` contract and not the particular treasury, it might happen that the backer tries to refund it through the `CampaignInfo.burn` function (which is accessible to anyone), losing all the assets associated with the pledge. This way, the assets from it might get stuck in the treasury.

Consider implementing access control on the `CampaignInfo.burn` function to restrict burning to only calls from the treasury contract.

Update: Resolved at [commit 7f6ed46](#).

L-03 Inconsistent Colombian Tax Accounting

In the `KeepWhatsRaised` contract, during creator withdrawals through the `withdraw` function, the Colombian creator tax path is executed when the `isColombianCreator` flag is enabled, and the tax is computed from the `availableBeforeTax` variable using the formula $\text{withdrawalAmount} * 40 / 10040$, which implies that the tax is already embedded in the `withdrawalAmount` variable.

However, in the [partial-withdrawal branch](#) the computed Colombian tax is added to the `totalFee` variable and then accounted for twice through the `s_availablePerToken[token] < (withdrawalAmount + totalFee)` check and the subsequent `s_availablePerToken[token] -= (withdrawalAmount + totalFee)` update, while the recipient still receives the full `withdrawalAmount` transfer. This creates a deviation where the Colombian tax is treated both as embedded in the withdrawal amount and as an extra additive fee. As a result, the platform fee accrual can become inflated, the available

balance can be reduced more than intended, and withdrawals can revert in cases where funds would otherwise be sufficient.

Consider enforcing a single accounting model for the Colombian tax in the `withdraw` function so that the `withdrawalAmount` variable is either treated as gross-including-tax or net-before-tax, but never both, preventing the Colombian tax from being more than it should be. Furthermore, in case Colombian legislation excludes the fees as part of expenses to the creator, consider adjusting the calculation so only the net amount (after fees are taken) is taxed with the Colombian fee. Moreover, as other countries might have similar tax formulas, consider refactoring the logic so it could be applied to countries besides Colombia. Furthermore, due to the precision of the Colombian Peso, the calculated tax might need to be rounded up to the next Peso to comply with the law. Therefore, consider rounding up to the next Colombian Peso.

Update: Resolved at [commit 04aa2ba](#). The colombian tax is now branched depending on if it will be included in the value or subtracted from the balance. It is worth mentioning that the naming used for the [new variables state "columbian"](#) when in reality it should be "colombian". Moreover, the rounding up effect applied is not nominated in COP but on the underlying token units, meaning that depending on the decimals and value, it might be more or less than 1 COP. Lastly, depending on the values of the flat fees and the available balances for the final withdraw, it is possible that the rounding up could cause a reversion due to lack of available funds, causing that some units of tokens could get stuck in the contract.

L-04 BaseTreasury.disburseFees Can Be Called Multiple Times

The abstract contract `BaseTreasury`'s `disburseFees` function can be called multiple times. Since this version of `disburseFees` function charges based on the [total balance times a fee percentage](#), this function could be used to drain the contract. Even though the contract is abstract and child contracts inheriting it prevent this from happening in practice, for instance in the [AllOrNothing.disburseFees function version](#), the `BaseTreasury` contract could be used in another future treasury which might not override its `disburseFees` function.

To avoid issues where fees can be charged multiple times, consider checking the `s_feesDisbursed` variable in the `BaseTreasury.disburseFees` function, instead of relying on the contracts inheriting from this one. Note that calling `disburseFees` multiple times is not possible as currently implemented, but it is a likely error in the future. If made possible by a future extension of `BaseTreasury`, the loss of funds would be nearly unlimited.

Update: Resolved at [commit 398679f](#).

L-05 Withdraw Can Happen Before Refund Window Ends

In the `KeepWhatsRaised` contract, the `claimRefund` function is intended to be only callable during the refund period computed by the `_checkRefundPeriodStatus` function using the deadline and the `refundDelay` value.

The `withdraw` function has a similar time window in which it cannot be called, `deadline + withdrawalDelay`. As both functions depend on the balance of the `s_availablePerToken` variable, if the `refundDelay` is bigger than the `withdrawalDelay` value, the `claimFund` function becomes callable before the refund window ends, allowing the `s_availablePerToken` variable to be drained and making refunds impossible.

Consider enforcing limitations for both delays so that the windows do not overlap.

Update: Resolved at [commit 4e7e6ca](#).

L-06 setFeeAndPledge Has No Time Restrictions

The `KeepWhatsRaised` function `setFeeAndPledge` has no time restrictions applied to it. For comparison, the `pledgeForAReward` and `pledgeWithoutAReward` functions both have the `currentTimeIsWithinRange` modifier. Although the `setFeeAndPledge` function is `onlyPlatformAdmin`, this is insufficient to protect it from being called after the deadline. The admin may make mistakes, be compromised, or experience delays during times of high congestion on the blockchain.

Consider applying the same `currentTimeIsWithinRange(getLaunchTime(), getDeadline())` modifier to `setFeeAndPledge`.

Update: Resolved at [commit bd5dd74](#).

L-07 Deselecting Platforms Skips `platformData` Setting

The `updateSelectedPlatform` function allows `selecting or deselecting a platformHash`. When `deselecting a platformHash`, all changes to `s_platformData[]` are skipped. This renders the input variables `platformDataKey` and `platformDataValue` useless, as they are only used within `if(selection)... branch`. It also results in the old data associated with the platform persisting and being `accessible via the getPlatformData function`.

Consider creating an `else` branch that allows for clearing elements of `s_platformData` associated with that `platformHash` which is being deselected only. The `platformDataKey` and `platformDataValue` input parameters can be used for this purpose in case `selection == false`.

Update: Acknowledged, not resolved. Team's statement for the issue:

Platform data keys are shared across platforms: any campaign can use any valid key, regardless of which platform introduced it. The `s_platformData` mapping is a flat key-value store and does not associate keys with a specific `platformHash`. Because of that, when deselecting a platform we cannot safely determine which keys to clear without risking data still used by other selected platforms. Clearing keys based on `platformDataKey` would require the caller to know which keys are exclusive to the deselected platform; if they clear a shared key, they could break other platforms. The current implementation avoids that by not clearing on deselect. So, the `platformDataKey` and `platformDataValue` arguments are ignored when `selection == false` for this reason.

L-08 Unescaped `s_imageURI` Can Produce Invalid `tokenURI` JSON Metadata

In the `CampaignInfo` contract, NFT metadata fields are initialized during the `initialize` function through a call into the `_initializeNFT` function in the `PledgeNFT` contract. At this point, `only the nftName input is validated` by the `_validateJsonString` function before assignment, while the `nftImageURI` input is stored `directly in the s_imageURI variable` without equivalent validation. The same deviation is present later in the campaign lifecycle when the owner calls `the setImageURI function`, which updates the `s_imageURI` variable without validation before launch.

This becomes problematic when the `tokenURI` function is called during metadata retrieval, because the `s_imageURI` variable is interpolated directly into an on-chain JSON string without escaping. Under the condition that the provided image URI includes JSON-breaking characters such as `"` or `\`, the generated metadata can become malformed and fail to parse by indexers, marketplaces, wallets, or other NFT consumers. This can cause pledge NFTs to display incorrectly or become effectively unusable in integrations that require valid ERC-721 metadata JSON.

Consider validating all user-controlled strings that are embedded in JSON, including the image URI at both initialization and update paths.

Update: Resolved at [commit ceb7c78](#).

L-09 Rewards Can Be Duplicated

The `reward` parameter in `_pledgeForAReward` and `pledgeForAReward` is never checked for duplicate entries. As such, it is possible that users can be overcharged by paying for the same reward multiple times.

Consider checking input arrays for duplicate entries. Alternatively, if duplicate entries are desired functionality, consider removing the check `rewardLen > s_rewardCounter.current()` from the `pledgeForAReward` and `_pledgeForAReward` functions.

Update: Resolved at [commit 8a4f8d3](#). The team stated that "Duplicate rewards are allowed by design. However, we agree with the issue of `rewardLen > s_rewardCounter.current()` and intend to remove that check". Nevertheless, the issue was later reintroduced in [commit cfa314e](#) reaching the [final version of the code](#).

L-10 Rewards Can Be Removed

The `removeReward` function of the `AllOrNothing` and `KeepWhatsRaised` contracts allows for deleting a configured reward at any time. This could cause errors if a reward that has already been pledged for is removed.

Consider implementing a flag that prevents the deletion of rewards if they have already been pledged for. Alternatively, consider removing this function or restricting it to before the beginning of the campaign so that unavailable rewards cannot be pledged for.

Update: Resolved at [commit 8f2034f](#).

L-11 Rewards for Pledge May Conflict

When a user selects rewards for a pledge within the `pledgeForAReward` function of the `AllOrNothing` contract or the `pledgeForAReward` function of the `KeepWhatsRaised` contract, they pass the rewards in as an array. One requirement is that the first reward must have `isRewardTier == true`. The rest of the rewards are considered "add-ons" on top of the first one, meaning they can optionally be included and may or may not be "reward tiers". However, this structure may create an issue where two incompatible rewards are included within the same list. For example, a reward that implicitly includes another reward. In such a case, a user may over-pay, as all reward values are summed.

Consider implementing a flag within the `Reward` struct for "canBeAddOn", and ensuring that for all elements after the first in `reward[]`, the `canBeAddOn` flag is `true`.

Update: Partially resolved at [commit fff7fdd](#). A new `canBeAddOn` flag was added to the `Reward` struct, and rewards selected after the first position in the pledge array are now required to have `canBeAddOn == true`. However, since all rewards must have a non-zero `rewardValue`, there is no on-chain mechanism to prevent a backer from selecting an add-on whose items semantically overlap with the primary tier, meaning that the values are always summed, so over-payment from incompatible reward combinations might remain possible.

L-12 setFeeAndPledge Combines Different Flows

The `setFeeAndPledge` function of the `KeepWhatsRaised` contract is intended for the platform admin to call to make pledges on behalf of some other account. The "fee" referenced in its name is the gateway fee, which is intended to be paid only when this function is used for making pledges. However, there are some issues with this function. For example, since gateway fees are based on `pledgeId`, and `pledgeId`s can be shared across accounts, it is possible that some non-gateway user can be charged a gateway fee. Additionally, the function requires setting a fee every time it is called. Finally, even when the gateway fee is `0`, useless fee calculations are still performed, which results in wasted gas.

Consider changing the `setFeeAndPledge` flow such that fees are not required to be set with every call. Consider using a per-gateway mapping which can be read as-needed, and implementing a `pledgeOnBehalfOf` function which the admin can call, passing in a gateway identifier. This will reduce gas usage and prevent users creating pledges on-chain from being charged fees by using fee-charging `pledgeId`s. Additionally, consider skipping gateway fee calculations when a gateway fee is not being charged.

Update: Acknowledged, not resolved. Team's statement for the issue:

Thanks for the suggestion. In our case, the gateway fee is not a fixed percentage and can vary based on external factors, making it more complex to determine on-chain. Because of this, the fee needs to be provided as an input at the time of calling the function rather than being derived from a stored per-gateway mapping. Additionally, the fee can be zero for certain platforms, depending on the specific integration.

L-13 Old Project Name Used for Storage Slots

In several parts of the codebase, a [reference to "ccprotocol"](#), the former name of the protocol, can be found. This name is being used as part of the hashing operation for ERC-7201 storage slots. However, as the name of the protocol is now "Oak Network", all of these instances should be updated to reflect it, and at the same time all the slot hashes should be re-calculated with the new string.

References can be found in:

- [AdminAccessCheckerStorage.sol](#)
- [CampaignInfoFactoryStorage.sol](#)
- [GlobalParamsStorage.sol](#)
- [TreasuryFactoryStorage.sol](#)

Consider changing both the documentation and calculating the new hashes in these and any other instance to reflect the new name of the protocol.

Update: Resolved at [commit 960c5c3](#).

L-14 Lack of Checks When Setting `campaignData`

Within the [CampaignInfo.initialize\(\)](#) function, the `s_campaignData` structure is set without any checks on the data within it. These values can also be set through special admin functions, which contain checks, such as the [updateLaunchTime](#), [updateDeadTime](#), and [updateGoalAmount](#) functions. These checks are necessary because they determine the timing of actions within various contracts in the system. In particular, because these are indirectly used by treasuries, such as the [TimeConstrainedPaymentTreasury](#) contract in [several of its operations](#).

To avoid contract malfunctions due to admin errors, consider applying checks to all elements of `campaignData` within the [initialize function](#).

Update: Acknowledged, not resolved. The client stated:

At the moment, our design assumes that all deployments go through the factory, where these validations are already enforced. We don't currently have plans to support deployments outside of the factory flow.

L-15 Unreachable Branch

Within `BasePaymentTreasury.sol`, the `if (duration == 0)` check in line 344 is impossible to be reached because `duration` is a `direct cast of maxExpirationBytes` above. If `maxExpirationBytes` is cast to `0`, the `if check in line 338` would evaluate to `true`, and `function execution would return and end`. Therefore, `the condition in line 344` will always evaluate to `false`, and the `entire if branch` can be removed.

Consider removing the unreachable code to make the codebase easier to understand and more gas efficient.

Update: Resolved at [commit 171f4f6](#).

L-16 Missing Check in `_updateFiatTransaction`

The `_updateFiatTransaction` function of the `FiatEnabled` contract appears to be designed to store transactions by ID and `accumulate the fiatRaisedAmount value`. However, there is no check on whether the `s_fiatAmountById element` has already been written to. If a previously used `fiatTransactionId` is used again, the accumulation on the `s_fiatRaisedAmount` variable will continue but the previous `s_fiatAmountById element` will be overwritten, meaning that it could get off-sync between the different tracking variables.

While the `FiatEnabled.sol` file appears to be unused, nonetheless, consider ensuring that `s_fiatAmountById[fiatTransactionId]` has no stored value within `_updateFiatTransaction` before updating the storage.

Update: Acknowledged, will resolve. Team's statement for the issue:

We accept the finding: reusing a `fiatTransactionId` overwrites `s_fiatAmountById` but still increases `s_fiatRaisedAmount`, so they can get out of sync. We will require that `s_fiatAmountById[fiatTransactionId]` is not already set before updating (or otherwise prevents id reuse). `FiatEnabled` is currently unused; we will still apply this fix. We will follow up with the concrete change.

L-17 `configureTreasury` Can Be Called Multiple Times

The `configureTreasury` function of the `KeepWhatsRaised` contract performs some checks on input parameters, but nothing prevents it from being called multiple times.

Since this function [affects sensitive system parameters of the treasury](#), consider implementing a flag preventing this function from being called more than once. Otherwise, users may be misled by timing or fee structure for the campaign changing, potentially while the campaign is active.

Update: Resolved at [commit 4dc46a9](#).

L-18 Deprecated Platform Data Due to `removePlatformData` Cannot Be Performed After Platform Is Delisted

In the `GlobalParams` contract, when a platform admin wants to [remove the data associated to a platform](#), the `removePlatformData` function first checks [if the platform is listed](#).

However, if the contract owner [already delisted](#) the platform, then the data associated with the deprecated platform will not be able to be erased, and it will be kept in storage, possibly confusing the state of the platform in the protocol and increasing the attack surface of the parts of the code querying such data.

Consider removing the associated platform data when delisting the platform.

Update: Acknowledged, not resolved. Team's statement for the issue:

Thanks for flagging this. Platform data keys are meant to be shareable across platforms. A key introduced by Platform X can be used by campaigns on Platform Y. Validation only checks that the key is valid via `checkIfPlatformDataKeyValid`; there is no platform-scoping for usage. Any campaign can use any valid key, regardless of which platform introduced it. Because of this, we do not remove platform data when a platform is delisted. If we did, we would invalidate keys that other platforms still rely on and could break existing campaigns. If you think this introduces any specific attack vectors, we would be happy to review it.

L-19 delistPlatform Does Not Clear platformAdapter Mapping Entry

In the `GlobalParams` contract, when [delisting a platform](#), the `delistPlatform` function erases the data from a set of mappings. However, it does not remove the platform adapter from the [platformAdapter mapping](#), meaning that it will be still attached to a deprecated platform.

Consider removing the platform adapter as well when delisting the platform.

Update: Resolved at [commit d73e80d](#).

L-20 Storage Update Without Emitting Event

Within the [CampaignInfoFactory contract](#), the [updateImplementation function](#) [updates a sensitive state variable](#) without emitting an event.

Consider always emitting an event when updating sensitive storage values.

Update: Resolved at [commit c966bfa](#).

L-21 Possible to Add Same Token to acceptedTokens List Twice

The [initialize function of the CampaignInfo contract](#) creates the `s_acceptedTokens` array by iterating over the `acceptedTokens[]` input parameter. However, there is no check for duplicate entries in the `acceptedTokens[]` list. If `s_acceptedTokens` contains duplicate entries, it could compromise the internal accounting, potentially double-counting balances on those tokens.

Consider checking the [s_isAcceptedToken\[token\] value](#) for each `token` before pushing it to the `s_acceptedTokens` array to prevent having identical elements.

Update: Resolved at [commit 1c1d53d](#).

L-22 Fee Values Are Uncapped

When setting the [platform fee percent](#) and the [protocol fee percent](#) in the `GlobalParams` contract, there are no checks on the values, especially being percentages. When these are

used within the `withdraw` function of the `BasePaymentTreasury` contract, if these are bigger than the equivalent of 100 percent, they could exceed the `balance` on which fees are computed, causing the `withdraw` call to revert. Additionally, if the `protocolFeePercent` value is too high within `processCryptoPayment`, similarly it could cause an `underflow and revert` when computing `netAmount`.

Consider imposing reasonable limits to both of these fees, and any other percentage in the protocol, during the setup. Their value should be considerably below the `PERCENT_DIVIDER` value, as it represents 100 percent. Ensure that the sum of all fees is less than `PERCENT_DIVIDER` when setting them.

Update: Partially resolved at [commit a12b883](#). The `updateProtocolFeePercent` function does not implement the validation for the sum of the fees (`platformFeePercent` and `protocolFeePercent`) meaning that these could exceed the `PERCENT_DIVIDER`. Moreover, allowing the sum to be 100% would result in not having room for the creator earnings.

L-23 Not Checking `s_fundClaimed`

The `s_fundClaimed` flag in the `KeepWhatsRaised` contract is set to `true` when the platform admin claims the treasury's tokens. This call also sets `s_availablePerToken[token]` to 0, and as the flag is not used in other user-accessible functions, further actions making use of the `s_availablePerToken` balance would probably revert due to the underflow operation and not due to an explicit `s_fundClaimed` check on a stage on which funds have already been claimed.

Consider utilizing the `s_fundClaimed` flag to quickly detect if refunds cannot occur. Additionally, consider applying the `s_fundClaimed` check to all logic which relies on knowing whether the admin has claimed the fund, signaling that certain operations cannot happen after such action.

Update: Resolved at [commit ec617a1](#).

L-24 `_msgSender` Overloading in Internal ID System Could Cause Accounting Errors

The `_msgSender` function allows the `_trustedForwarder` to specify the address to be treated as `msg.sender`. In the event the `trustedForwarder` fails or is compromised, the `_msgSender` function returning `address(0)` may create confusion for payments, which

`reserve address(0)` for off-chain payments. In the event the trusted forwarder fails, a call to `processCryptoPayment` may store 0 for the `s_paymentIdToCreator` element corresponding to this payment. This will corrupt accounting and store an on-chain payment as an off-chain one.

Moreover, the implementation of the `_findPaymentId` function of the `BasePaymentTreasury` contract lacks the robustness to differentiate this mistake, as the search is being done sequentially on both fronts, off-chain and then on-chain, using the `_msgSender` value. This means that if a payment is accidentally classified as a different type, subsequent functions using such output might get stuck as each payment type has different restrictions (in particular the `buyerId` and `buyerAddress` values).

Consider adding a sanity check to `_msgSender` within `BasePaymentTreasury` contract and `BaseTreasury` contract which reverts if the `sender == address(0)` in the cases where it should not. Moreover, consider removing the variable overloading in the sender and correctly specifying the type of payment being queried to improve the robustness of the `_findPaymentId` function.

Update: Resolved at [commit e8cc973](#).

L-25 Unbounded Arrays As Inputs

Throughout the codebase, there are many instances of accepting arrays as direct user inputs, with no maximum limits on their length. Arrays that are too large can cause out-of-gas errors and may cause certain flows to get stuck.

For instance, the `createPayment` function of the `BasePaymentTreasury` contract accepts a user-input `lineItems` array. While it should be noted that only the `platform admin` can call this function, the line items are intended to match a user's off-chain purchase and thus the line items can be assumed to match it as well. A sufficiently large `lineItems` array may cause this function to be inoperable.

Additionally, while the `createPayment` function call `iterates over lineItems once`, those line items are then stored and loaded again within the `confirmPayment` function and looped over twice, once `within the _calculateLineItemTotals` function and again within the `_updateLineItemsForConfirmation` function. For a large enough array, it is likely that `createPayment` will succeed while `confirmPayment` fails and reverts for some payment. This breaks the flow of the contract and will lead to stuck funds.

Consider implementing a maximum length for input arrays on `external` or `public` functions. Note that the example highlighted is not the only instance of unbounded user-input arrays.

Update: Partially resolved at [commit ba0b13b](#). Limitations to the arrays have been imposed in the `BasePaymentTreasury` contract. However, no caps have been imposed in the other contracts, such as the `AllOrNothing` or `KeepWhatsRaised` contract.

L-26 Shipping Fee Not Verified

In the `AllOrNothing` contract, the `pledgeForAReward` function includes a `shippingFee` parameter. This parameter is used within the `_pledge` function to transfer funds from the `backer`. However, since this parameter is never verified, its value can be anything desired by the caller, who is the end user. Thus, end users will be incentivized to choose `0` as their `shippingFee`.

Since the parameter is intended to be used to manage fees for shipping physical rewards, a zero value is intuitively incorrect. Consider implementing a check that `shippingFee != 0` or that the creator must define such value beforehand and the `backer` needs to supply it for the selected pledge.

Update: Acknowledged, not resolved. Team's statement for the issue:

We do not consider this a protocol-level issue in the current design. `shippingFee` is intentionally caller-supplied and is not enforced on-chain because shipping costs can vary by fulfillment context and may be determined off-chain by the creator/platform. A zero shipping fee is therefore not inherently invalid: some rewards may have free shipping, digital fulfillment, included shipping, creator-subsidized shipping, or no shipping at all. More generally, if a backer submits a lower or higher shipping fee than expected, that does not create a fund extraction issue against the contract itself; it reflects the amount the backer chose to pay as part of that pledge. Any required shipping amount, if applicable, is expected to be handled by the platform's off-chain checkout/order logic. So we do not think a blanket check such as `shippingFee != 0` would be correct. If stricter enforcement is desired, that would need to come from a different design where shipping fees are predefined per reward / region / fulfillment rule and validated against on-chain configuration. In the current model, however, zero shipping fee is valid by design.

L-27 Platform Data Passed Might Not Equal Amount of Platforms Passed

Within the `initialize` function of `CampaignInfo.sol`, matching `arrays of platformDataKey` and `platformDataHash` are checked against their length in the `CampaignInfoFactory` contract. These are `iterated over`, setting an element of `s_platformData` with the corresponding key and value pair. Similarly, the `selectedPlatformHash` input is also `iterated and stored` in the `s_isSelectedPlatform` mapping.

However, there is no check that the length of the `selectedPlatformHash` input matches with the `platformDataKey` input. This means that if the length does not match between both inputs, a platform might not have its data or that data stored relates to none of the platforms. More importantly, there is no linkage between the platform and its data that guarantees that certain data belongs to a certain platform and vice versa.

Consider enforcing the length between the platform and the data and defining some linkage between the data and the platform.

Update: Acknowledged, not resolved. The team's statement for the issue:

We reviewed this under our intended design and do not plan to enforce equality between `selectedPlatformHash.length` and `platformDataKey.length`. `CampaignInfo` stores platform selection as a set (`s_isSelectedPlatform`) and stores configuration as a global key $\rightarrow$ value map (`s_platformData`), not as a per-platform row aligned by array index with `selectedPlatformHash`. The same data key is therefore allowed to be shared across platforms; the number of selected platforms and the number of initial key-value entries are independent dimensions (including the valid case of zero initial data keys with one or more selected platforms). We already enforce `platformDataKey.length == platformDataValue.length` in `CampaignInfoFactory` before initialization, so there is no indexing hazard between keys and values. Requiring the platform array length to match the data array length would incorrectly constrain shared-key campaigns and would not reflect how consumers resolve data (via `getPlatformData(key)`, not via position next to a platform hash). We acknowledge that correct behavior depends on off-chain / integrator discipline when constructing the initial key list and values; we accept that tradeoff rather than encoding a false 1:1 “platform slot $\leftrightarrow$ data slot” invariant on-chain.

L-28 Missing Initializer Lock in Treasury Implementations

During treasury setup, the `TreasuryFactory` contract registers implementation addresses and later uses the `deploy` function to create minimal proxy instances that [call the `initialize` function](#). In that flow, implementation contracts are expected to remain inert and permanently uninitialized while stateful operation is delegated to clones. However, the `PaymentTreasury` contract, the `TimeConstrainedPaymentTreasury` contract, the `AllOrNothing` contract, and the `KeepWhatsRaised` contract define empty constructors and expose the `initialize` function under the `initializer` modifier, while the `BaseTreasury` contract and the `BasePaymentTreasury` contract do not disable initializers in a constructor. This leaves each implementation instance directly initializable by any external account once the implementation address is discoverable, which can be used by an attacker to mislead users.

Consider adding a constructor in the `BaseTreasury` contract and the `BasePaymentTreasury` contract that calls `_disableInitializers()` so that all inheriting treasury implementations are locked by default.

Update: Resolved at [commit 0974aa8](#).

L-29 Lack of Documentation About the `backer`

The `PledgeNFT` contract makes use of the OpenZeppelin `ERC721Burnable` contract which uses the `_safeMint` function to mint the respective pledge to the `backer`. As the function will check the receiving end with the `ERC721Utils.checkOnERC721Received` method, to be able to use this protocol, it is expected that the `backer` is either an EOA or a contract/EIP-7702 account that implements the `onERC721Received` hook.

However, due to the lack of documentation and enforcement, the `backer` address might not fulfill these requirements. Additionally, if only EOAs should be allowed, an enforcement of `backer.code.length == 0` before mint should be done. Moreover, the usage of the `ERC721Utils.checkOnERC721Received` method opens the possibility to increase the attack surface, as an attacker might have a way to initiate a reentrancy call if the implementation of the `onERC721Received` hook returns the call to the protocol, which might be exploitable in the future.

Consider thoroughly documenting the requirements of the `backer` to let users know about possible reversions of the pledge. Additionally, consider protecting the external actions against reentrancy initiated during any of the affected pledges.

Update: Resolved at [commit 75faa27](#).

L-30 Platform Data Keys Are Not Bound to the Updated Platform During Selection Updates

In the `CampaignInfo` contract, during pre-launch owner configuration through the `updateSelectedPlatform` function, the caller provides one `platformHash` that is expected to represent the specific platform being updated together with its platform data updates. In this flow, the `GlobalParams` contract validates each provided key only through the `checkIfPlatformDataKeyValid` function and then persists values into the `s_platformData` variable.

The function does not verify that each provided key is owned by the same platform represented by the `platformHash` variable, even though ownership metadata is tracked in the `GlobalParams` contract and exposed through the `getPlatformDataOwner` function. This deviation can occur whenever the owner passes valid keys that belong to another listed platform, because global key validity is accepted independently of key underlying ownership. As a result, updating one selected platform can overwrite data intended for a different platform, which breaks platform data isolation at campaign level and can cause silent misconfiguration, inconsistent off-chain interpretation, and incorrect downstream behavior that depends on these values.

Consider enforcing that keys belong to the respective `platformHash`.

Update: Partially resolved at [commit 35df4e2](#). A similar effect can still happen during the `createCampaign` function.

L-31 Platform Adapter Rotation Does Not Revoke the Trusted Forwarder in Deployed Treasuries

The protocol supports ERC-2771-style meta-transactions through platform adapter contracts. The `deploy` function of the `TreasuryFactory` contract reads the adapter once via the `getPlatformAdapter` function of the `GlobalParams` contract and passes it to treasury initialization. Each treasury stores the adapter in `_trustedForwarder` variable during

initialization (`BaseTreasury.__BaseContract_init` and `BasePaymentTreasury.__BaseContract_init`) and uses it in the `_msgSender` function (`BaseTreasury._msgSender` and `BasePaymentTreasury._msgSender`) to determine the effective caller.

However, rotating the adapter through a call to the `setPlatformAdapter` function from the `GlobalParams` contract does not update already-deployed treasuries, and `_msgSender` does not consult `GlobalParams` at runtime. As a result, the old adapter remains a permanently trusted forwarder for those treasuries. If the old adapter is compromised or malicious, it can continue impersonating privileged roles gated by `_msgSender` and keep performing platform-admin actions despite the adapter rotation.

Consider adding a controlled mechanism to update or revoke `_trustedForwarder` in deployed treasuries (for example, callable by the protocol admin), or redesigning `_msgSender()` to validate forwarded calls against the `GlobalParams` contract dynamically. Consider also documenting adapter immutability if rotation is not intended to secure existing treasuries.

Update: Resolved at [commit a0b63c7](#).

L-32 Missing Validation

In the `CampaignAccessChecker` contract, there are no validations being done over the `campaignInfo` input in the `__CampaignAccessChecker_init` function nor the functions that end up calling this one (starting from the `TreasuryFactory` contract). That means that not only a mistakenly placed address could be assigned as the `CampaignInfo` contract, but also it would allow a platform admin to place a malicious contract as the `CampaignInfo` contract and alter the flow of operations and/or funds.

Consider using introspection via [ERC-165](#) to prevent accidentally linking the wrong contracts, and consider documenting the risks of whitelisting platform admins.

Update: Resolved at [commit 6c06c87](#).

L-33 Inconsistent Use of `paymentId`

In the `BasePaymentTreasury` contract, there is an inconsistent use of the `paymentId`. Inside the `claimRefund` function, there are cases where the

`PaymentTreasuryPaymentNotClaimable` custom error reverts with the `internalPaymentId`, and sometimes it reverts with the `paymentId`.

Consider always using the same type to be consistent during failed transactions.

Update: Resolved at [commit 9fcc553](#).

L-34 Mismatch in Fee Storage

Within the `disburseFees` function, there are two fees applied. One is `PLATFORM_FEE_PERCENT`, which is stored during `BaseTreasury` initialization, and one is `INFO.getProtocolFeePercent()`, which is fetched from `CampaignInfo` during execution.

Since the `PLATFORM_FEE_PERCENT` is a "snapshot" value while the protocol fee may change over time, consider changing one or the other so they are both consistent (in that they are snapshots, or they are fetched in real-time). If this mismatch is intentional, consider documenting the reasoning.

Update: Resolved at [commit f15865c](#).

L-35 Treasury Self-Pledge Misuses NFT Check

When pledging in the `AllOrNothing` and `KeepWhatsRaised` treasuries, the implementations allow arbitrary callers to provide an arbitrary `backer` to then execute a `safeTransferFrom(backer, address(this), amount)` before minting pledge NFTs and increasing `raised-amount` accounting.

If the accepted ERC-20 implementation allows a `transferFrom(owner, ...)` call without allowance when `msg.sender == owner` (a common "self-spend" shortcut), an attacker can set `backer` and `tokenSource` to the treasury itself, causing no real asset inflow while still minting NFT pledges and increasing internal balances. These fake pledges can be created without new funds, inflating `s_tokenRaisedAmounts` raised accounting.

Current versions of the `AllOrNothing` and `KeepWhatsRaised` contracts do not possess the `onERC721Received` hook, which means that the minting process would revert during the hook. However, as such hook is meant to protect against using wallets not supporting such assets, relying on the lack of the hook to mitigate the possible inflation of raised totals, refund and fee disbursement reversions, and corrupt metrics, is not a strong invariant.

Consider rejecting the treasury address as the payer (referred to as `backer` and `tokenSource`) in pledge paths in both `AllOrNothing` and `KeepWhatsRaised` to avoid accounting contamination. Moreover, consider measuring the treasury token balance before and after transfers, and using the actual received amount for state updates.

Update: Resolved at [commit e7d38fe](#). Backer cannot be the same treasury and delta balance changes have been implemented. However, the delta is not compared against the `totalAmount` transferred, but against the `shippingFeeInTokenDecimals` meaning that pledges can still pass a bigger `totalAmount` and the validation will not mitigate the problem if the value received is equal or greater than `shippingFeeInTokenDecimals`. Same situation with the `tip` case. Later, at [commit 73163a6](#) and also during the fixes implementing the `Permit2` functionality, the deltas were removed as rebasing or fee-on-transfer tokens will not be accepted from the admin side.

Notes & Additional Information

N-01 Redundant Conditionals

Throughout the codebase, multiple instances of redundant conditionals were identified:

- [Line 500 of CampaignInfo.sol](#) checks `deadline` against `launchTime` and `launchTime + minimumCampaignDuration`. These can be reduced to a check that `deadline < launchTime + minimumCampaignDuration`.
- [Line 523 of CampaignInfo.sol](#) checks `deadline` against `launchTime` and `launchTime + minimumCampaignDuration`. These can be reduced to a check that `deadline < launchTime + minimumCampaignDuration`.
- [Line 1253 of BasePaymentTreasury.sol](#) contains the conditional `availablePaymentAmount < amountToRefund`, which will always trigger reverts if the [check in line 1306 succeeds](#). Since the check in line 1306 is more strict, the *second* condition on line 1253 is unnecessary.

Consider resolving the redundant checks for greater clarity and more gas efficiency.

Update: Resolved at [commit 11f6498](#).

N-02 Reverts With Multiple Conditions

Within the codebase, there are many instances of multiple conditions being combined which lead to the same revert:

- [Lines 602-603 of BasePaymentTreasury.sol](#)
- [Lines 683-685 of BasePaymentTreasury.sol](#)
- [Lines 709-710 of BasePaymentTreasury.sol](#)
- [Lines 793-794 of BasePaymentTreasury.sol](#)
- [Lines 58-59 of CampaignInfoFactory.sol](#)
- [Line 222 of AllOrNothing.sol](#)
- [Lines 228-229 of AllOrNothing.sol](#)
- [Lines 286-287 of AllOrNothing.sol](#)
- [Line 519 of KeepWhatsRaised.sol](#)
- [Line 612 of KeepWhatsRaised.sol](#)
- [Lines 618-619 of KeepWhatsRaised.sol](#)
- [Lines 752-753 of KeepWhatsRaised.sol](#)
- [Line 977 of KeepWhatsRaised.sol](#)
- [Line 1070 of KeepWhatsRaised.sol](#)

Each reversion should be triggered by a single condition, or a small set of interlinked conditions. The custom error message should make the cause of the revert extremely clear, such that the user does not need to troubleshoot their transaction using a block explorer or other tooling.

Update: Resolved at [commit 4f3ebda](#).

N-03 Unnecessary Temporary Variables

Throughout the codebase, multiple instances of unnecessary variable declarations were identified:

- Within the `if` clause in [line 882 of BasePaymentTreasury.sol](#), the variable `feeAmount` is used only once, and then deleted and re-declared the next time it is needed. Consider removing this variable, and refactoring the `if` clause such that `protocolFee` is computed, and its effects are applied only when `applyProtocolFee` is true.
- Within the `_calculateLineItemTotals` function, the `countsTowardsGoal`, `applyProtocolFee`, and `instantTransfer` booleans are unnecessary as they are

only used once. Consider removing them and instead directly reading these values when needed.

Consider implementing the above suggestions to save gas and improve the maintainability of the codebase.

Update: Resolved at [commit 751d6d4](#).

N-04 Unnecessary Assignment in `AllOrNothing._pledge`

Within the `_pledge` function of the `AllOrNothing` contract, the `shippingFeeInTokenDecimals` is calculated. In the case where `reward == ZERO_BYTES` (the `else` case), this is only triggered when the call to `_pledge` comes from the `pledgeWithoutAReward` function, in which case the `shippingFee` is set to `0`.

Since the `shippingFeeInTokenDecimals` variable is `0` upon declaration, consider removing the superfluous assignment to it in [line 422](#) to save gas and simplify the codebase, or assigning the value of "0" to assert such value.

Update: Resolved at [commit 9181723](#).

N-05 `external` Function With `internal` Naming Convention

In the `CampaignInfo` contract, the `_setPlatformInfo` function is an `external` function, as it is meant to be called from the `deploy` function in the `TreasuryFactory` contract. In addition, the `_pauseCampaign`, `_unpauseCampaign`, and `_cancelCampaign` functions are also `external`, designed to be called by protocol admins or campaign owners.

However, these function names start with an underscore, which `internal` functions usually use when following de-facto naming conventions. Consider removing the underscore in the functions' names to follow the naming convention of the rest of the codebase.

Update: Resolved at [commit ff0a799](#).

N-06 Multiple Event Emissions for Same Assignment

The `addItemBatch` function from the `ItemRegistry` contract does not validate if the elements passed to it are unique, meaning that the `ItemAdded` event will be emitted multiple times for the same element, polluting the off-chain indexers.

Consider validating that each element is unique before assigning and emitting the event.

Update: Resolved at [commit 7cbe940](#).

N-07 `addItem` And `addItemBatch` Functions Can Modify Existing Items

In the `ItemRegistry` contract, the `addItem` and `addItemBatch` functions are used to add new items to the creator's list. However, as there are no checks on the items being added, the creator can set mapping elements to zero, effectively deleting them. Additionally, they can overwrite previously set items in the `Items` mapping, potentially misleading users about the properties of `Items`.

Consider adding checks to ensure that previously added items are not being overwritten. In addition, consider adding a `removeItem` function if removal capabilities are desired.

Update: Resolved at [commit 9d290ec](#).

N-08 Non-Reward Pledges Are Not De-Normalized

When a user makes a pledge for a reward, the `pledgeAmount` input variable is de-normalized within the `_pledge` functions in the `KeepWhatsRaised` and the `AllOrNothing` contracts. However, when a user pledges without a reward, the `pledgeAmount` is not denormalized. The amount, expressed in token units, is then used to [transfer funds](#), so this denormalization is necessary.

Consider modifying the call to `_pledge` function within the `AllOrNothing` contract's `pledgeForAReward` and `KeepWhatsRaised` contract's `pledgeForAReward` functions to pass `pledgeAmounts` in that are already denormalized. This helps standardize the behavior of `_pledge` function for further extension, and is less error-prone. Additionally, consider

documenting the `_pledge` functions with NatSpec comments explaining the expected units for `pledgeAmount`, to avoid user error.

Update: Partially resolved at [commit 770564e](#). The denormalization has been moved into the `pledgeForAReward` function and more documentation has been added to the `_pledge` function. However, when pledging without a reward, the flow relies on the user to pass already denormalized values, and the [documentation for the `pledgeWithoutAReward` function](#) does not state that it needs to be denormalized already. Moreover, the issue has been reintroduced in the [final version](#) of the code.

N-09 Unnecessary Override

Throughout the codebase, multiple instances of unnecessary overrides were identified:

- The `burn` function of the `CampaignInfo` contract does not change the implementation compared to the [one it is overriding \(i.e., the `burn` function of the `PledgeNFT` contract\)](#). To simplify the code and make it easier to understand, consider removing the unneeded `burn` function in the `CampaignInfo` contract or documenting the reason behind it.
- The `cancelled` function of the `BaseTreasury` contract overrides the one from the `PausableCancellable` contract, while only [calling it via `super`](#). Since this does not add any functionality and is already visible to the `BaseTreasury` contract, consider removing the overriding `cancelled` definition in `BaseTreasury` contract.

Consider addressing the above to improve the clarity and maintainability of the codebase.

Update: Resolved at [commit 13905c8](#).

N-10 Renaming Opportunities

Throughout the codebase, multiple opportunities for better naming were identified.

- The `s_refundableNonGoalLineItemPerToken`, `totalNonGoalClaimableAmount`, and `totalNonGoalRefundableAmount` variables do not match in their ordering of words. Consider defining a standard order of words within long variable names to make them easier to track.
- Two `claimRefund` functions exist with different purposes. Consider renaming one to make it easier to understand the difference and prevent incorrect calls.

- `s_paymentIdToToken` and `s_paymentIdToTokenId` are extremely similar in name. Consider renaming `s_paymentIdToTokenId` to `s_paymentIdToNFTId` to make the difference clearer.
- `__checkTimeWithinRange` and `__checkTimeIsGreater` do not specify what range or what target time must be greater than. Consider renaming them to reflect their boundaries more clearly.
- The `platformScopedKey` variable in `__getMaxExpirationDuration` is unclear in its purpose as the expiration duration key. Consider renaming it to `expirationDurationKey`.
- The `__scopePaymentIdForOnChain` and `__scopePaymentIdForOffChain` functions can be renamed to better link them to the concept of "internal" payment IDs which frequently appear in the contract. Consider renaming these functions to `__getInternalPaymentIdForOnChain` and `__getInternalPaymentIdForOffChain` to reduce confusion on the concept of internal payment IDs.

Consider addressing the above to improve the clarity and maintainability of the codebase.

Update: Partially Resolved at [commit c8d5244](#). `claimRefund` function not renamed.

N-11 Unnecessary Assignment Wastes Gas

The assignment of `amount` to `withdrawalAmount` inside the `else` statement in the `withdraw` function of the `KeepWhatsRaised` contract is unneeded. This is because the `amount` could be used directly for the comparisons in the subsequent `if` statements, and the `withdrawalAmount` value could be used instead of calling the `s_availablePerToken[token]` several more times in said `if` statements.

Consider reusing the available local variable and input to improve the gas usage.

Update: Resolved at [commit fbd5186](#).

N-12 Unclear Docstrings

Throughout the codebase, multiple instances of unclear docstrings were identified:

- In [line 16 of AllOrNothing.sol](#), the docstring is generic, and does not explain how the `AllOrNothing` file differentiates from other crowdfunding designs within the codebase.

- In [line 49 of CampaignInfo.sol](#), the docstring states "O(1) token validation". This provides little explanation of how token validation is performed.
- In [line 1032 of BasePaymentTreasury.sol](#), the docstrings do not specify which balance is being checked.

Consider addressing the above instances of unclear docstrings to improve the clarity of the codebase.

Update: Resolved at [commit e86feb5](#).

N-13 Not Using Require With Custom Errors

Since [Solidity 0.8.26](#), support for custom errors in `require` statements has existed.

Throughout the codebase, there are many instances of using `if` statements followed by `revert`s with errors. For example, [within the updateSelectedPlatform function](#).

Consider upgrading the Solidity version and replacing this pattern with `require( <condition>, <error> )` for clearer and more efficient code.

Update: Acknowledged, not resolved. The team's statement for the issue:

We have decided not to migrate to solc 0.8.26 and adopt `require(condition, CustomError())` at this time. Applying that pattern across the codebase would mean a large, mechanical refactor of all existing `if / revert` guards. Some of which do not map cleanly to a single `require` (e.g. non-trivial control flow or multiple related conditions), which introduces a non-zero risk of subtle mistakes during migration due to the sheer volume. Given that, we do not believe the marginal clarity or gas characteristics of this style justify the migration effort and regression risk at this time, and we prefer to keep the current explicit `if / revert` pattern for the time being.

N-14 Unchecked Scoping Not Needed for `for` Loop Iterators

Since [Solidity 0.8.22](#), the `unchecked` keyword applied to `for` loop iterator increments is no longer needed to reduce the gas consumption. For example, in [line 656 of `BasePaymentTreasury.sol`](#), the `unchecked` block can be removed and replaced with a simple `i++` increment in the `for` loop. The same occurs in lines [266](#), [269](#) and [602](#) of [GlobalParams.sol](#), as well as line [762](#), [772](#), [910](#), and [1217](#) of [BasePaymentTreasury.sol](#).

Consider removing the `unchecked` block for greater clarity and moving the increment operator to the `for` loop statement.

Update: Resolved at [commit `f8cb375`](#).

N-15 Structs Declared in Middle of Contracts

The [Solidity Style Guide](#) states that type declarations should be declared first in a contract body, before state variables, events, errors, modifiers, or functions.

The [LineItemTotals](#) struct is declared after functions in [BasePaymentTreasury](#). Similarly, the [Config](#) struct is declared after functions within the [CampaignInfo](#) contract.

Consider moving these declarations to the beginning of the contract to follow the Solidity Style Guide.

Update: Resolved at [commit `f3eb641`](#).

N-16 Implicit Returns and Unnamed Return Values

Throughout the codebase, multiple instances of implicit returns or unnamed return values were identified.

- The [_getMaxExpirationDuration](#) function uses an implicit return pattern if execution reaches the end of the function. Consider instead doing an explicit `return` statement.

- The `getRaisedAmount`, `getAvailableRaisedAmount`, `getLifetimeRaisedAmount`, `getRefundedAmount`, and `getExpectedAmount` functions do not have named return values, despite being more computationally intense than [simple "getter" functions](#). Consider naming the return values of these functions and adding NatSpec comments defining their values.

Generally, ensure that all return values are named, to show their purpose, and all returns are explicit using the `return` keyword to avoid accidentally returning intermediate or undefined values.

Update: Resolved at [commit b9c5b44](#).

N-17 Missing Documentation

The [parameters for `\_\_BaseContract\_init`](#) are not documented.

Although this function is not `external` or `public`, it is important enough that the inputs should be well-defined. As such, consider documenting the parameters following [NatSpec formatting](#).

Update: Resolved at [commit f63b4ab](#).

N-18 Unused Code

The `whenCancelled` modifier in `PausableCancellable` is unused.

Consider removing the unused code for improved clarity and maintainability of the codebase.

Update: Acknowledged, not resolved. The team's statement for the issue:

We acknowledge that the `whenCancelled` modifier is currently unused. `PausableCancellable` is intended as a small shared abstraction for pause/cancel semantics; keeping `whenNotCancelled` and `whenCancelled` as a symmetric pair matches the intended lifecycle (active vs cancelled) and mirrors the pause side (`whenNotPaused` / `whenPaused`). We therefore prefer to retain `whenCancelled` for API completeness and possible future use (e.g. post-cancellation-only entry points), rather than removing it for cleanliness alone.

N-19 Repeated Use of Errors

Error messages assist developers and users in debugging transactions, so they should be unique and specific to avoid confusion between cases. It is encouraged to use distinct error messages for each instance in the codebase, to make it clear exactly where execution failed.

Throughout the codebase, multiple instances of the same error messages being used for different error cases were identified:

- The `AllOrNothingNotClaimable` error is used if [the time is incorrect or the campaign is not successful](#), and if the `amountToRefund == 0`.
- The `PaymentTreasuryPaymentNotClaimable` error is used [frequently throughout BasePaymentTreasury](#) for various reasons.
- The `AllOrNothingInvalidInput` error is used many times in `AllOrNothing.sol`, for example if [array lengths in inputs don't match](#) and if [rewards are not defined](#).

Please note that this list is not exhaustive, and that there are many error messages which are re-used throughout the codebase.

Consider implementing distinct error messages for each case. Consider using the style of error message exemplified by the [Cancelled error of PausableCancellable.sol](#), where a unique `reason` string is included with the error. This can be leveraged to use the same error definition with a unique error code.

Update: Resolved at [commit acd726c](#). Implementation was later changed at [commit 1617830](#) and [commit 8c07af9](#) to resolve a bytecode size limitation, where enums were introduced for errors. It is worth noting that in the `amountToRefund == 0` condition for the `claimRefund` functions in the `BasePaymentTreasury` contract do not use the same error code. Moreover, insertions in the enum have been made, instead of appending new elements at the end, which might cause unexpected behaviors when decoding with older enum declaration.

N-20 Typographical Error

In line 100 of `AllOrNothing.sol`, the docstring should say "Emitted within disburseFees" instead of "Emitted when disburseFees".

Consider correcting the identified instance of typographical error for improved code clarity.

Update: Resolved in commit [54ada6b](#).

Conclusion

In the course of this audit, many issues were identified across the codebase, ranging from critical-severity to note-severity. While the codebase needed extra complexity to handle different campaign styles and the real-world nature of crowdfunding, rewards, and off-chain payments, it was found to be exceedingly complicated and stands to considerably benefit from simplification. Given the size and complexity of the codebase, along with the number of issues identified during the review, it is reasonable to assume that additional findings may surface with further analysis. The volume and nature of the issues uncovered suggest that continued review would likely provide additional security and robustness improvements.

For this reason, we strongly recommend conducting a re-audit after the fix review has been completed. This is particularly important if substantial refactoring is undertaken in response to the findings, as structural changes can introduce new risks and warrant a fresh, comprehensive assessment.

While audits can impact timelines and require dedicated resources, they are critical to ensuring a secure, reliable, and well-defined protocol. We are happy to provide guidance on prioritizing which components should be included in a follow-up review, as well as on best practices to improve overall code quality in preparation for the next audit.

Improving code clarity and structure will directly enhance the effectiveness of future security assessments. We strongly encourage implementing all recommended fixes, adhering to established Solidity style guidelines, and thoroughly documenting the codebase. This process often reveals additional edge cases and latent bugs. Expanding test coverage, particularly around edge cases and complex logic, will further strengthen the system and reduce risk ahead of a re-audit.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.